

Competitive Programming

Section 55 Editorial: Randomization And Hashing

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Equal Substrings	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Palindrome Substrings	5
3.1	Restatement	5
3.2	Observations and algorithm	5
3.3	Correctness proof	5
3.4	Complexity	5
3.5	Full C++ solution	5
3.6	Full Python solution	5
4	C. Anagram Substrings	6
4.1	Restatement	6
4.2	Observations and algorithm	6
4.3	Correctness proof	6
4.4	Complexity	6
4.5	Full C++ solution	6
4.6	Full Python solution	6
5	What to carry forward	7
6	Complete upsolve notes	8
6.1	A. Equal Substrings	8
6.2	B. Palindrome Substrings	8
6.3	C. Anagram Substrings	8

1 Editorial overview

1.1 Problem-specific proof map

A. Equal Substrings. Equal strings have equal normalized polynomial hashes; the local double-hash implementation makes accidental equality negligible, and all queries are answered from prefix hashes in $O(1)$.

B. Palindrome Substrings. A substring is palindromic precisely when its forward hash equals the aligned reverse-string hash. Normalization compares the same exponent range.

C. Anagram Substrings. Prefix counts for each letter make the count vector of any substring a subtraction. Two strings are anagrams exactly when all their letter counts agree.

1.2 Detailed solution guide

A. Substring Equality. The definition asks whether two substrings have the same characters in the same order. The local exact solution compares them directly. The scalable version stores prefix hashes so the hash of $[l, r]$ can be extracted in $O(1)$ and normalized by length. Equal hashes are treated as evidence of equality; double hashing reduces collision risk.

B. Palindrome Hashing. A substring is a palindrome iff it equals its reverse. With hashes, compare the forward hash of $[l, r]$ in s with the corresponding forward hash in $\text{reverse}(s)$. The direct local implementation compares mirrored characters, which proves the same condition without collision concerns.

C. Anagram Queries. Two substrings are anagrams iff each character appears the same number of times in both. Prefix frequency arrays let us compute the 26-count vector for any substring by subtraction. Comparing those vectors is an exact deterministic hash for lowercase letters.

Common pitfalls. Hash indices are easy to shift by one, so write the substring formula once and test length-one substrings. Randomized hashing is probabilistic; frequency vectors for small alphabets are exact.

The local exercises use exact checks when they are cheap and prefix summaries when they are better.

- A compares equal-length substrings by their stored representation.
- B compares mirrored characters, which is exact for the local constraints; the lesson shows how forward/reverse hashes replace it at scale.
- C uses 26 prefix frequency arrays, which is deterministic and collision-free for lowercase anagram queries.

Hashing is presented as an optimization path, not as a substitute for a proof of what is being compared.

2 A. Equal Substrings

2.1 Restatement

Answer whether two same-length substrings are equal.

2.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

2.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

2.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(
```

2.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().split(); s=lines[0]; q=int(lines[1]); idx=2; out=[]
    for _ in range(q):
        l1,r1,l2,r2=map(int,lines[idx:idx+4]); idx+=4; out.append("YES" if s[l1-1:r1]==s[l2-1:r2])
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Palindrome Substrings

3.1 Restatement

For each query, say whether the first substring is a palindrome.

3.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

3.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

3.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(
```

3.6 Full Python solution

```
import sys
def main():
    lines=sys.stdin.read().split(); s=lines[0]; q=int(lines[1]); idx=2; out=[]
    for _ in range(q):
        l1,r1,l2,r2=map(int,lines[idx:idx+4]); idx+=4; out.append("YES" if s[l1-1:r1]==s[l1-1:r2])
    print("\n".join(out))
if __name__=="__main__": main()
```

4 C. Anagram Substrings

4.1 Restatement

Answer whether two same-length substrings have the same multiset of letters.

4.2 Observations and algorithm

The proof map above states the invariant or reduction for this task. The implementation mirrors that argument: every branch is one of its explicit mathematical cases.

4.3 Correctness proof

The task-specific invariant in the proof map holds at initialization and is preserved by every stated transition. The final state therefore represents the requested answer. $\square$

4.4 Complexity

The bound is the concrete one stated for this task in the proof map; the code implements exactly its enumerations, transitions, or queries.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!c
```

4.6 Full Python solution

```
import sys,collections
def main():
    lines=sys.stdin.read().split(); s=lines[0]; q=int(lines[1]); idx=2; out=[]
    for _ in range(q):
        l1,r1,l2,r2=map(int,lines[idx:idx+4]); idx+=4; out.append("YES" if sorted(s[l1-1:r1])
    print("\n".join(out))
if __name__=="__main__": main()
```

5 What to carry forward

6 Complete upsolve notes

6.1 A. Equal Substrings

Build powers and prefix hashes. Normalize each same-length substring hash by subtracting its preceding prefix times the matching power; equal hashes answer YES probabilistically. The prefix recurrence proves the extraction formula. Preprocess $O(n)$, query $O(1)$; double hash in adversarial settings and test prefix/suffix/full ranges.

6.2 B. Palindrome Substrings

Build the same hash for s and reversed s . Compare $[l, r)$ against reversed $[n-r, n-l)$; equality is the palindrome condition. Preprocess $O(n)$, query $O(1)$. Test length one, even/odd palindrome, and an off-by-one reverse boundary.

6.3 C. Anagram Substrings

For lowercase input, use 26 prefix counts, not a randomized hash. Two ranges are anagrams iff every character-count difference agrees. Preprocess $O(26n)$ or 26 tables and query $O(26)$; test repeated letters, unequal counts, and full ranges. This is exact and avoids collision reasoning.

In C++ use long long modular multiplication; Python should avoid slicing in the fast path. Stress all three with direct string comparisons on short random data.

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.